

清华大学电子工程系本科生核心课

数据与算法 - 第4讲

优先级队列、串、串匹配

冯伟 (fengwei@tsinghua.edu.cn)

2024年10月8日





- 优先级队列
- 串：概念与ADT、存储表示
- 字符串的模式匹配

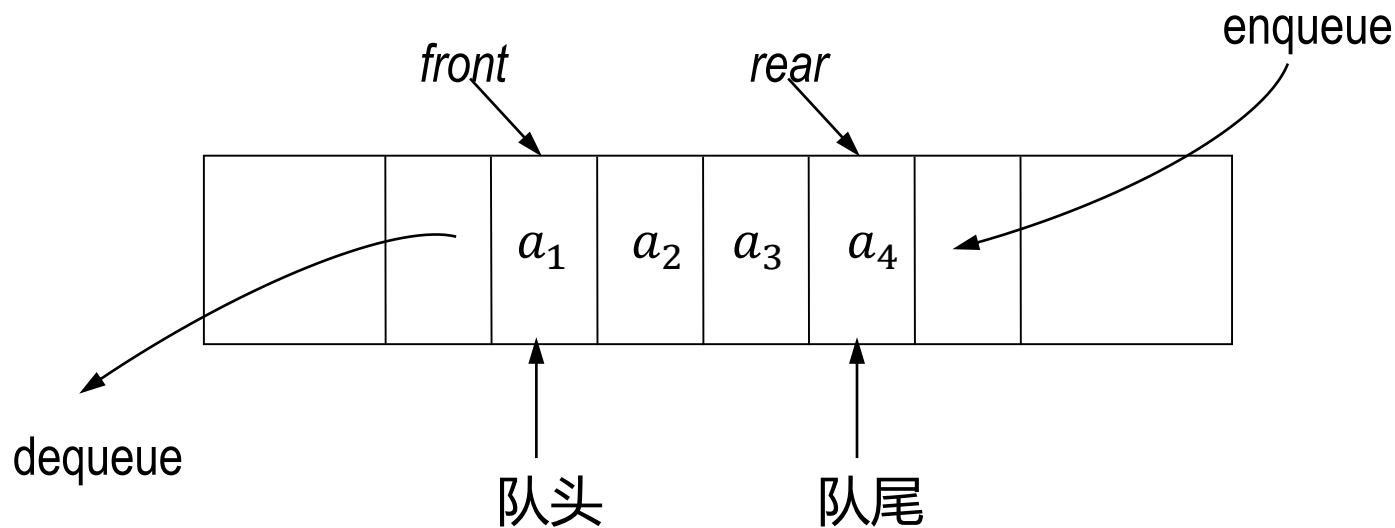


- 优先级队列
- 串：概念与ADT、存储表示
- 字符串的模式匹配



优先级队列 (Priority Queue)

- 队列是限定在表的一端进行插入，而在另一端进行删除的线性结构
- 队列是先进先出的线性表

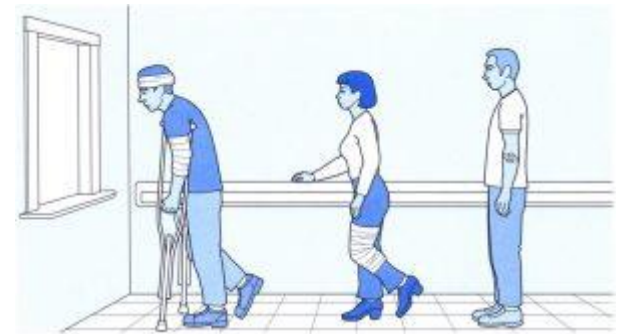


- 优先级队列：队列中每个元素都增加一个代表其优先级的权值，根据权值访问优先级最高的元素

优先级队列 (Priority Queue)

- 不同于一般的先进先出队列，每次从队列中取出的是具有最高优先级的元素

任务序号	1	2	3	4	5
优先级	20	1	40	30	10
执行顺序	3	5	1	2	4



- 计算机根据程序优先级，来决定处理各程序的先后次序；在处理过程中，允许优先级较高的程序中断优先级较低的程序
- 根据优先级来决定执行顺序是生活中的常用策略



优先级队列 – 抽象数据类型

ADT PQueue{

数据对象: $D = \{a_i \mid a_i \in ElemSet, i = 1, 2, \dots, n, n \geq 1\}$

数据关系: $R1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i = 2, \dots, n \}$

基本操作:

InitPQueue(&PQ)

操作结果: 构造一个空的优先级队列PQ

DestroyPQueue(&PQ)

初始条件: 优先级队列PQ已存在

操作结果: 优先级队列PQ被销毁

ClearPQueue(&PQ)

初始条件: 优先级队列PQ已存在

操作结果: 将PQ清为空队列



优先级队列 – 抽象数据类型

IsEmpty(PQ)

初始条件：优先级队列PQ已存在

操作结果：若PQ为空队列，则返回TRUE，否则返回FALSE

PQueueLength(PQ)

初始条件：优先级队列PQ已存在

操作结果：返回PQ的元素个数，即优先级队列的长度



优先级队列 – 抽象数据类型

GetFront(PQ, &e)

初始条件：优先级队列Q存在且非空

操作结果：用e返回优先级PQ中优先级最高的元素

PQueueInsert(&PQ, e)

初始条件：优先级队列PQ已存在

操作结果：把元素e插入到优先级队列PQ中

PQueueDelete(&PQ, &e)

初始条件：优先级队列Q存在且非空

操作结果：删除优先级队列PQ中优先级最高的元素，并用e返回其值

} ADT PQueue



优先级队列 – 实现方式

- 基于 有序 顺序表 的实现
- 基于 无序 顺序表 的实现
- 基于 有序 链表 的实现
- 基于 无序 链表 的实现

对“优先级”而言

对“队列”而言



优先级队列 – 顺序表实现

```
class CListPQ{
private:
    int    *queue;                //队列指针
    int    queueLen, MaxLen;      //队列长度和最大规模
public:
    CListPQ(int len) {queue = (int *)new int[len]; MaxLen = len; queueLen = 0;}
    ~CListPQ()       {delete queue; queueLen = 0; MaxLen = 0;}
    void    clearPQ() {queueLen = 0;}                //清空队列
    bool    IsEmpty() {return queueLen == 0;}        //队列判空
    int     PQLength() {return queueLen;}            //队列长度
    void     getFront(int &e);                       //取优先级最高的元素
    void     PQInsert(int e);                         //插入操作
    void     PQDelete(int &e);                       //删除操作
    void     Change(int i, int e);                   //修改元素优先级
};
```



优先级队列 – 无序顺序表实现

- 队列中的元素并不需要按照优先级高低的顺序排列
 - 执行入队操作时不关心待插入元素的优先级，直接放在队尾
 - 时间复杂度 $O(1)$

```
void CListPQ::PQInsert(int e) {  
    queue[queueLen] = e;  
    queueLen ++;  
}
```



优先级队列 – 无序顺序表实现

- 队列中的元素并不需要按照优先级高低的顺序排列
 - 访问优先级最高的元素，需要遍历队列中的所有元素
 - 时间复杂度为 $O(N)$

```
void CListPQ::getFront(int &e){  
    int e = queue[0];  
    for(int i = 1; i < queueLen; i++)  
        {if(e < queue[i]) e = queue[i];}  
}
```

- 改变队列中某个元素的优先级，时间复杂度： $O(1)$



优先级队列 – 无序顺序表实现

□ 出队操作：首先找到具有最高优先级的元素，然后将其删除

□ 时间复杂度为 $O(N)$

```
void CListPQ::PQDelete(int &e){  
    int p = 0;  
    for(int i = 1; i < queueLen; i++)  
        { if(queue[p] < queue[i])    p = i;}  
    e = queue[p];  
    for(i = p ; i < queueLen-1; i++)  
        queue[i] = queue[i+1];  
    queueLen --;  
}
```

//p为优先级最高元素的位置

//返回删除元素值

//移位保持“顺序存储”



优先级队列 – 有序顺序表实现

➤ 队列中元素始终按照优先级从高到低的顺序排列

□ 入队操作需要把新元素根据其优先级插入到队列中合适位置上

□ 时间复杂度为 $O(N)$

有更好的方法吗？

```
void CSListPQ::PQInsert(int e){
```

```
    int i = queueLen - 1;
```

```
    while(i >= 0){
```

```
        if(queue[i] < e) {queue[i+1] = queue[i];}
```

```
        else{queue[i+1] = e; break;}
```

```
        i--;
```

```
    }
```

```
    if(i < 0) queue[0] = e;
```

```
    queueLen ++;
```

```
}
```

//自后向前寻找合适位置

//边找边移动

//找到位置，插入元素

//特例：应插入在队头位置

//长度加一



优先级队列 – 有序顺序表实现

- 有序顺序表取最高优先级元素，时间复杂度 $O(1)$

```
void CSListPQ::getFront(int &e) {e = queue[0];}
```

- 出队操作：时间复杂度为 $O(N)$ 如果把最高优先级元素放在队尾呢？

```
void CSListPQ::PQDelete(int &e){  
    e = queue[0];                                //删除元素位于队头  
    for(int i = 0; i < queueLen-1; i++)          //移位操作  
        {queue[i] = queue[i+1];}  
    queueLen--;                                  //长度减一  
}
```

- 改变队列中某个元素的优先级，时间复杂度: $O(N)$



优先级队列 – 链表实现

链表结点定义

```
typedef struct INode{  
    int    data;                //数据元素  
    INode *next;               //后向指针  
    INode(int e) {data = e; next = NULL;} //已知元素的构造函数  
    INode(int e, INode *p) {data = e; next = p;} //已知元素和指针的构造函数  
}LNODE;
```




优先级队列 – 链表实现

```
class CLinkPQ{
    private:
        LNODE *head;                //链表头指针
    public:
        CLinkPQ() {head = NULL;}    //构造函数
        ~CLinkPQ() {}               //析构函数
        void      clearPQ();         //清空队列
        bool      IsEmpty() {return head == NULL;} //队列判空
        void      getFront(int &e);  //取高优先级元素
        void      PQInsert(int e);   //插入元素
        void      PQDelete(int &e);  //删除元素
        void      Change(LNODE *p, int e); //修改元素优先级
};
```



优先级队列 – 无序链表实现

□ 无序链表的插入，时间复杂度为 $O(1)$

```
void CLinkPQ::PQInsert(int e){  
    if(head)      head = (LNODE *)new LNODE(e, head);  
    else          head =(LNODE *)new LNODE(e);           //注意空队列情况  
}
```

□ 无序链表取最高优先级元素，时间复杂度为 $O(N)$

```
void CLinkPQ::getFront(int &e){  
    LNODE *p = head; e = p->data;  
    while(p){  
        if(p->data > e) e = p->data;  
        p = p->next;  
    }  
}
```



优先级队列 – 无序链表实现

- 无序链表的删除、出队操作，时间复杂度为 $O(N)$

```
void CLinkPQ::PQDelete(int &e){
    LNODE *p = head, *q = head;           //q指向待删除前序结点
    e = p->data;
    while(p->next) {                       //寻找优先级最高的元素
        if(p->next->data > e) {q = p; e = q->next->data;}
        p = p->next;                       //q为待出队结点的前序结点
    }
    if(e == head->data) {head = q->next; delete q;} //删除头结点
    else {p = q->next; q->next = p->next; delete p;} //删除非头结点
}
```

- 改变队列中某个元素的优先级，时间复杂度为 $O(1)$ (酌情分析)



优先级队列 – 有序链表实现

□ 入队操作：有序链表的插入，时间复杂度为 $O(N)$

```
void CSLinkPQ::PQInsert(int e){
    LNODE *node, *p = head;
    node = (LNODE *)new LNODE(e);           //构造新结点
    if(!head) {head = node; return;}        //case1: 队列为空
    if(head->data < e)                        //case2: 插入元素优先级最高
        {node->next = head; head = node; return;}
    while(p->next) {
        if(p->next->data < e)                //case3: 找到合适的插入位置
            {node->next = p->next; p->next = node; break;}
        p = p->next;                        //更新指针
    }
    if(!p->next) p->next = node;            //case4: 插入位置在队列尾
}
```



优先级队列 – 有序链表实现

- 有序链表取最高优先级元素，时间复杂度为 $O(1)$

```
void CSLinkPQ::getFront(int &e) {e = head->data;}
```

- 有序链表的删除，时间复杂度为 $O(1)$

```
void CSLinkPQ::PQDelete(int &e){
```

```
    LNODE *p = head;                                //有序链表第一个元素优先级最高
```

```
    e = p->data;
```

```
    head = p->next;
```

```
    delete p;
```

```
}
```

- 改变队列中某个元素的优先级，时间复杂度为 $O(N)$



优先级队列 – 实现方法比较

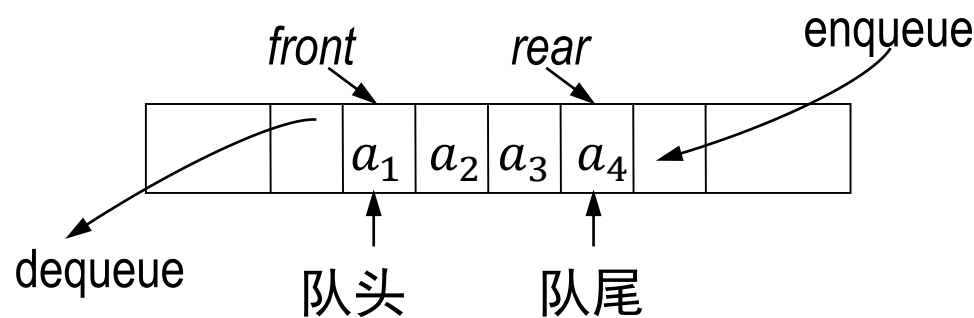
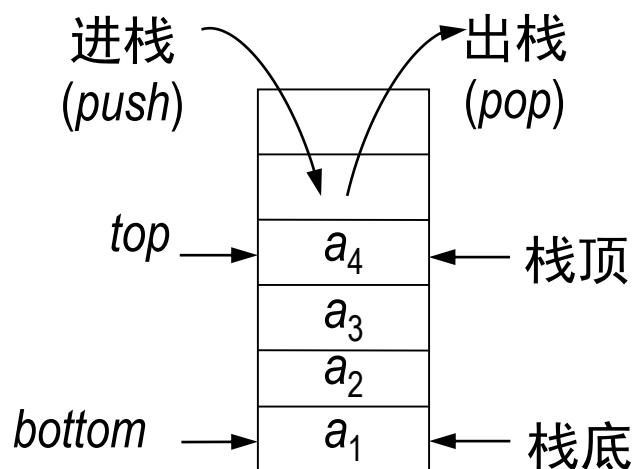
	插入	查找高优先级元素	删除	修改优先级
无序顺序表	$O(1)$	$O(N)$	$O(N)$	$O(1)$
有序顺序表	$O(N)$	$O(1)$	$O(N)$	$O(N)$
无序链表	$O(1)$	$O(N)$	$O(N)$	$O(1)$
有序链表	$O(N)$	$O(1)$	$O(1)$	$O(N)$

- 少量删除最大优先级操作，大量插入操作，宜采用无序表
- 大量查找操作，宜采用有序表



优先级队列、栈、队列的联系

- 栈：先进后出 $\longleftrightarrow$ 队列：先进先出
- 从优先级队列角度，根据元素在队列中的停留时间设定优先级
 - 栈首先删除停留时间最短的元素，时间越短权重越大
 - 队列首先删除停留时间最长的元素，时间越长权重越大



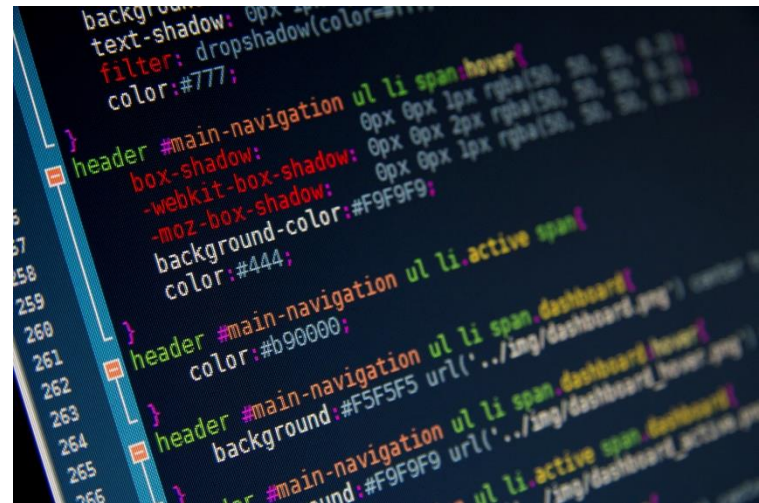
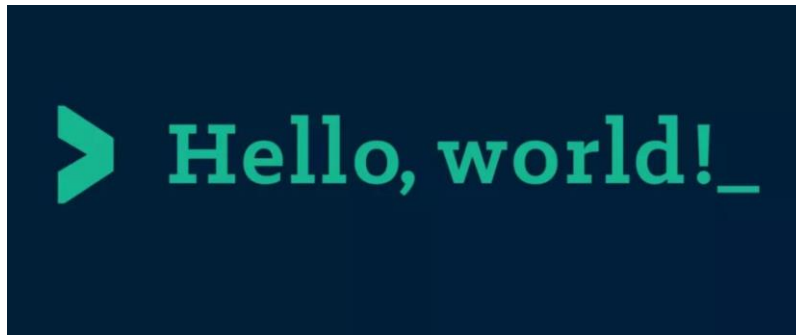


提纲

- 优先级队列
- 串：概念与ADT、存储表示
- 字符串的模式匹配

字符串 (String)

- $S = "a_0a_1 \cdots a_n"$
- 串是有限长度的字符序列，是计算机中最常用的数据类型之一
- 限定数据元素为字符的线性表





字符串 (String)

- 一个串的长度指的是串中所包含的字符个数
 - 当 $n = 0$ 时, 称 S 为空串, 记为 $S = \phi$
 - 值为空格的字符串 $\neq$ 空串: “ ” $\neq \phi$
 - 长度为1的字符串 $\neq$ 单个字符: “ a ” $\neq$ ‘ a ’
- 字符在串中的序号称为该字符在串中的位置
- 两个串相等当且仅当
 - 两个串的长度相等
 - 每个对应位置的字符也相同
 - 字符串地址不必相同 (可存储于不同的地址)



子串与主串

- 串中任意多个连续的字符组成的子序列称为该串的子串
- 包含该子串的串称为子串的主串
- 子串在主串中的位置指的是该子串的第一个字符在主串中的位置
- 串的操作经常以子串为单位进行，这是与线性表的区别



串的例子

$a = \text{"Data"}$

$c = \text{"DataStructure"}$

$b = \text{"Structure"}$

$d = \text{"Data Structure"}$

- a 的长度为4, b 的长度为9, c 的长度为13, d 的长度为14; $c \neq d$
- a 是 c 的子串 $a \subset c$, a 也是 d 的子串 $a \subset d$
- b 是 c 的子串 $b \subset c$, b 也是 d 的子串 $b \subset d$
- a 在 c 和 d 中的位置都是0
- b 在 c 的位置为4, b 在 d 中的位置为5
- c 和 d 非常相似, 但不相等



串的抽象数据类型 (ADT)

ADT String {

数据对象: $D = \{a_i | a_i \in CharacterSet, i = 0, 1, \dots, n-1, n \geq 1\}$

数据关系: $R = \{\langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i = 1, \dots, n-1\}$

基本操作:

StrAssign (&T, chars)

初始条件: chars是字符串常量

操作结果: 生成一个值等于chars的串T

StrEmpty (S)

初始条件: 串S已存在

操作结果: 若S为空串, 则返回TRUE, 否则返回FALSE



串的抽象数据类型 (ADT)

ClearString (&S)

初始条件：串S已存在

操作结果：将S清为空串

StrCompare (S, T)

初始条件：串S和T已存在

操作结果：若 $S > T$ ，则返回值 > 0 （比较字符的ASCII码）

若 $S = T$ ，则返回值 $= 0$

若 $S < T$ ，则返回值 < 0 （以第一个不相同字符的比较结果为准）

StrLength (S)

初始条件：串S已存在。

操作结果：返回S的元素个数，称为串的长度

SubString (&Sub, S, pos, len)

初始条件：串S已存在， $0 \leq \text{pos} \leq \text{StrLength}(S)-1$ 且 $0 \leq \text{len} \leq \text{StrLength}(S)-\text{pos}$

操作结果：用Sub返回串S的第pos个字符起长度为len的子串



串的抽象数据类型 (ADT)

Index (S, T, pos)

初始条件：串S和T已存在，T是非空串， $0 \leq \text{pos} \leq \text{StrLength}(S) - 1$ 。

操作结果：若主串S中存在和串T值相同的子串, 则返回它的主串S中第pos个字符之后第一次出现的位置; 否则函数值为-1

Replace (&S, T, V)

初始条件：串S,T和V已存在，T是非空串

操作结果：用V替换主串S中出现的所有与T相等的非重叠的子串

StrCopy (&T, S)

初始条件：串S已存在

操作结果：由串S复制得串T



串的抽象数据类型 (ADT)

StrCat (&T, S1, S2)

初始条件：串S1和S2已存在。

操作结果：用T返回由S1和S2联接而成的新串。

StrInsert (&S, pos, T)

初始条件：串S和T已存在， $0 \leq \text{pos} \leq \text{StrLength}(S)$ 。

操作结果：在串S的第pos个字符之后插入串T。

StrDelete (&S, pos, len)

初始条件：串S已存在， $0 \leq \text{pos} \leq \text{StrLength}(S) - \text{len}$ 。

操作结果：从串S中删除第pos个字符起长度为len的子串。

} ADT String



串的抽象数据类型 (ADT)

- 串类型的最小操作子集 (C语言中处理字符、字符串的库函数)
 - 串赋值 StrAssign
 - 串复制 StrCopy (strcpy) //将某地址字符串拷贝至他地址
 - 串比较 StrCompare (strcmp) //字符串比较
 - 求串长 StrLength (strlen) //计算字符串长度, 遇到\0结束
 - 串联接 StrCat (strcat) //字符串拼接
 - 求子串 SubString
- 其它串操作可以在最小操作子集的基础上实现
- C++中有string类, 可以用 = 进行赋值操作, == 进行比较, + 做串联



串的抽象数据类型 (ADT)

Index算法实现：在S中第pos个字符之后搜索T，返回位置

```
int Index(String S, String T, int pos){  
    n = StrLength(S);                //主串的长度  
    m = StrLength(T);                //子串的长度  
    i = pos;                          //搜索的起始值  
    while(i <= n-m) {  
        SubString(sub,S,i,m);        //从位置i取长度为m的子串  
        if(StrCompare(sub,T) ) i++;  //与待查找的子串T比较，不等后移一位  
        else return i;               //相等返回当前位置  
    }  
    if(i == n-m+1) return -1;  
}
```



串的存储

- 串的存储方式与一般线性表的存储方式类似
- 顺序存储
 - 定长顺序存储
 - 变长顺序存储
- 链式存储



串的定长顺序存储

- 顺序存储在一个指定大小的存储区域
- 预先分配了固定大小的存储区域
- 串的序列长度超过指定区域大小时，必须截断

$a =$

T	h	i	s		i	s		a						
---	---	---	---	--	---	---	--	---	--	--	--	--	--	--

$b =$

	s	h	o	r	t		s	t	r	i	n	g		
--	---	---	---	---	---	--	---	---	---	---	---	---	--	--

$a + b =$

T	h	i	s		i	s		a		s	h	o	r	t
---	---	---	---	--	---	---	--	---	--	---	---	---	---	---



串的变长顺序存储

- 根据实际需要分配存储区域的规模
- C语言中用 '\0' 作为字符串的终结符
- 避免了截断，提高了空间利用率

$a =$

T	h	i	s		i	s		a	\0
---	---	---	---	--	---	---	--	---	----

$b =$

	s	h	o	r	t		s	t	r	i	n	g	\0
--	---	---	---	---	---	--	---	---	---	---	---	---	----

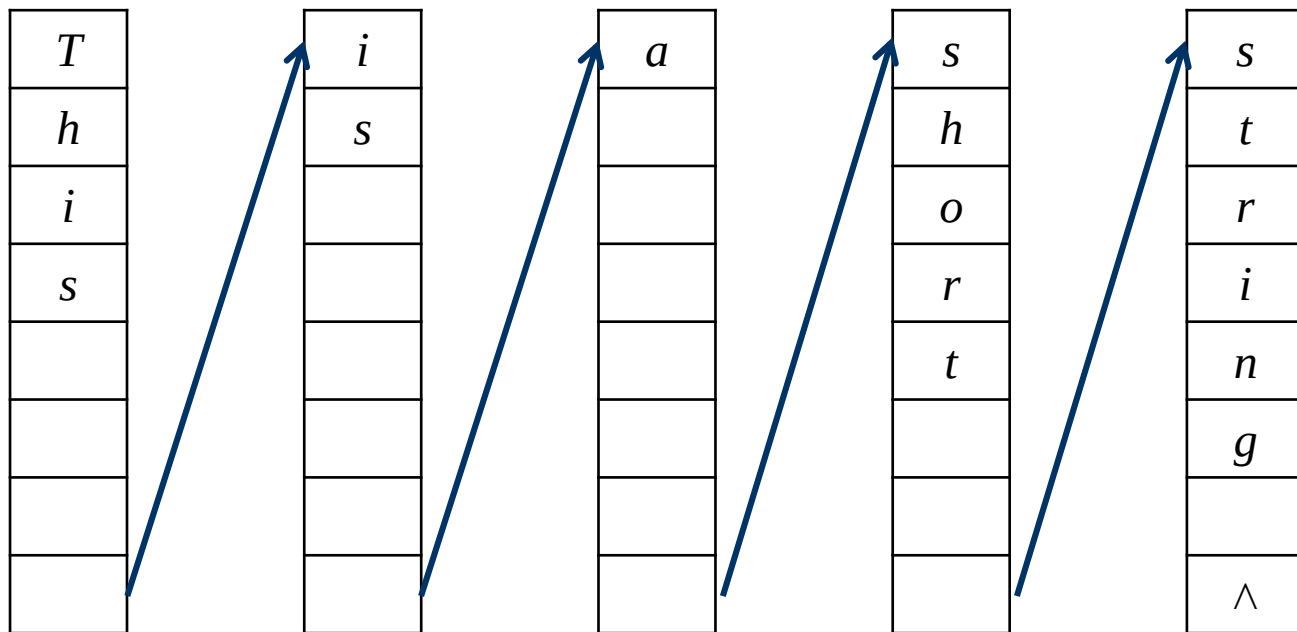
$a + b =$

T	h	i	s		i	s		a		s	h	o	r	t
s	t	r	i	n	g	\0								



串的链式存储（块链存储）

- 与一般的单链表类似
- 一个结点可以存储多个字符
- 通常把一个结点存储的字符个数称为结点大小





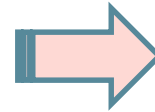
- 优先级队列
- 串：概念与ADT、存储表示
- 字符串的模式匹配



串的模式匹配

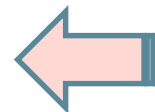
已知目标串T和模式串P，模式匹配就是要**在目标串T中找到一个与模式串P相等的子串 (子串定位)**

- 如果找到，匹配成功，返回模式串P在目标串T中位置
- 如果没有找到，匹配失败



串匹配算法的应用

- 文本编辑
- 程序调试
- 搜索引擎
- 病毒防范
- 生命科学





串匹配算法

目标串

<i>K</i>	<i>n</i>	<i>o</i>	<i>w</i>	<i>l</i>	<i>e</i>	<i>d</i>	<i>g</i>	<i>e</i>	<i>i</i>	<i>s</i>	<i>p</i>	<i>o</i>	<i>w</i>	<i>e</i>	<i>r</i>	<i>\0</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	-----------

模式串

<i>e</i>	<i>d</i>	<i>g</i>	<i>e</i>	<i>\0</i>
----------	----------	----------	----------	-----------

➤ 蛮力法 (Brute-Force)

➤ KMP算法



蛮力法

穷举匹配：把T的每个位置都作为可能的起始位置，与模式串P逐次进行比较，如相等，则匹配成功，终止；否则就一直比较下去，直到遍历目标串所有位置，匹配失败，过程终止

```
int BruteForceMatch(char *T, char *P){  
    int n = strlen(T);                //求目标串T的长度  
    int m = strlen(P);                //求模式串P的长度  
    int i, j;  
    for(i=0; i <= n-m; i ++){         //逐个试探目标串的位置  
        j = 0;  
        while(j < m && T[i+j] == P[j]) j++;  
        if(j == m) return i;          //与模式串比较  
                                        //在i处匹配成功  
    }  
    return -1;                        //匹配失败  
}
```



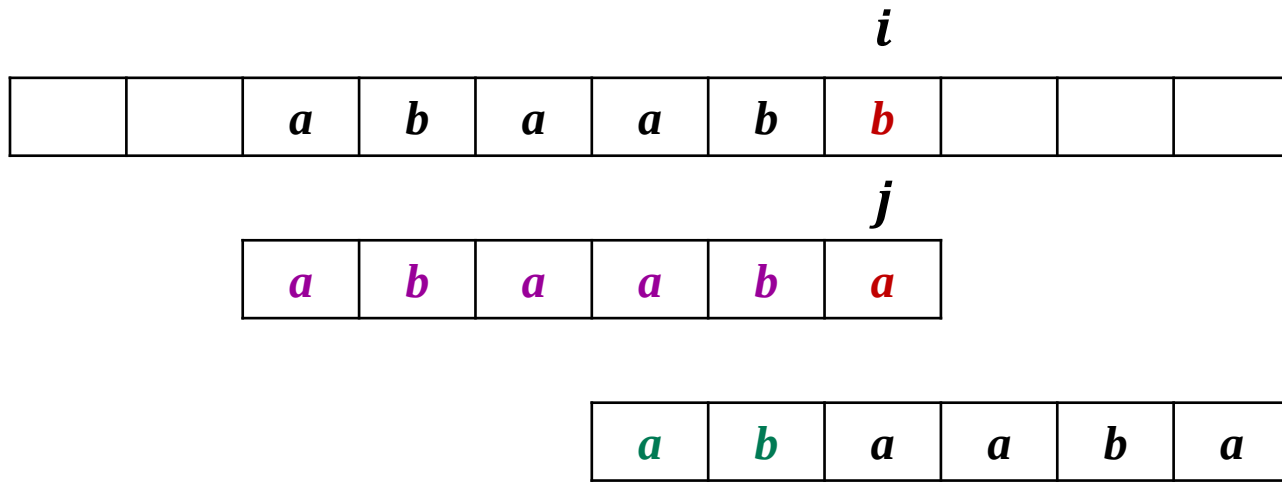
蛮力法

- 假设目标串 T 的长度为 n ，模式串 P 的长度为 m ，蛮力法最坏情况下的时间复杂度为 $O(n \times m)$
 - $T = \text{"aaaa} \cdots \text{ah"}$
 - $P = \text{"aaah"}$
- 优点：直观，实现简单
- 缺点：算法效率不高，只适用于解决小规模的模式匹配问题
- C++中string类的成员函数 `find()` 用的蛮力法吗？
- 蛮力法并没有考虑在比较过程中得到的信息



KMP算法

- Knüth-Morris-Pratt算法 (1977)
- KMP算法按自左向右的方向匹配
- 在匹配过程中，当模式P不匹配时，应利用在比较过程中得到的信息，尽量向右移动最大距离，以避免重复比较





KMP算法

i

		<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>b</i>			
--	--	----------	----------	----------	----------	----------	----------	--	--	--

j

<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>
----------	----------	----------	----------	----------	----------

<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>
----------	----------	----------	----------	----------	----------



<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>
----------	----------	----------	----------	----------	----------



<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>
----------	----------	----------	----------	----------	----------

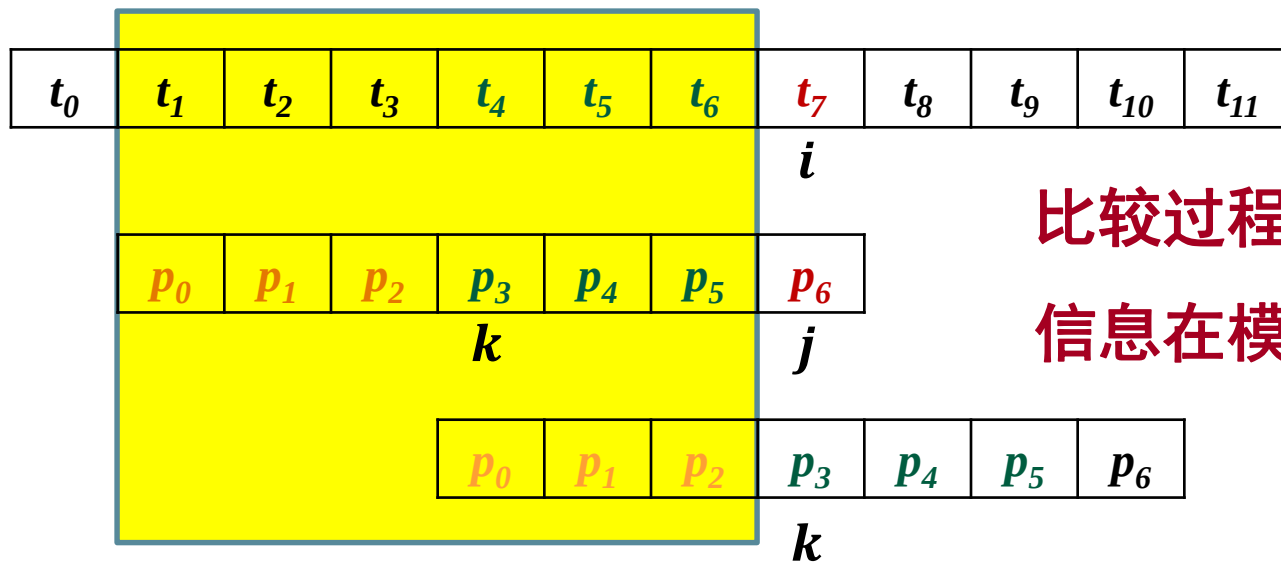


向右移动最大距离，避免重复比较，也不错过可能匹配的子串



KMP算法

- 目标串 $T = t_0 t_1 \dots t_n$ 模式串 $P = p_0 p_1 \dots p_m$
 - 不匹配位为 $t_i \neq p_j$
- 找最小的 k , 使得下一步可直接将 t_i 和 p_k 进行比较, 不会错过机会
 - 一方面有: $t_{i-k} t_{i-k+1} \dots t_{i-1} = p_{j-k} p_{j-k+1} \dots p_{j-1}$
 - 另一方面有: $t_{i-k} t_{i-k+1} \dots t_{i-1} = p_0 p_1 \dots p_{k-1}$
 - 因此有: $p_0 p_1 \dots p_{k-1} = p_{j-k} p_{j-k+1} \dots p_{j-1}$



比较过程中得到的
信息在模式串里面



KMP算法

➤ 步骤一：模式串 P 预处理

- **部分匹配串**：设 j 是模式串 P 的不匹配位置，在 $P[0, \dots, j-1]$ 中找出 $P[0, \dots, j-2]$ 的前缀中最长的一个，它恰好等于 $P[1, \dots, j-1]$ 的后缀中最长的一个

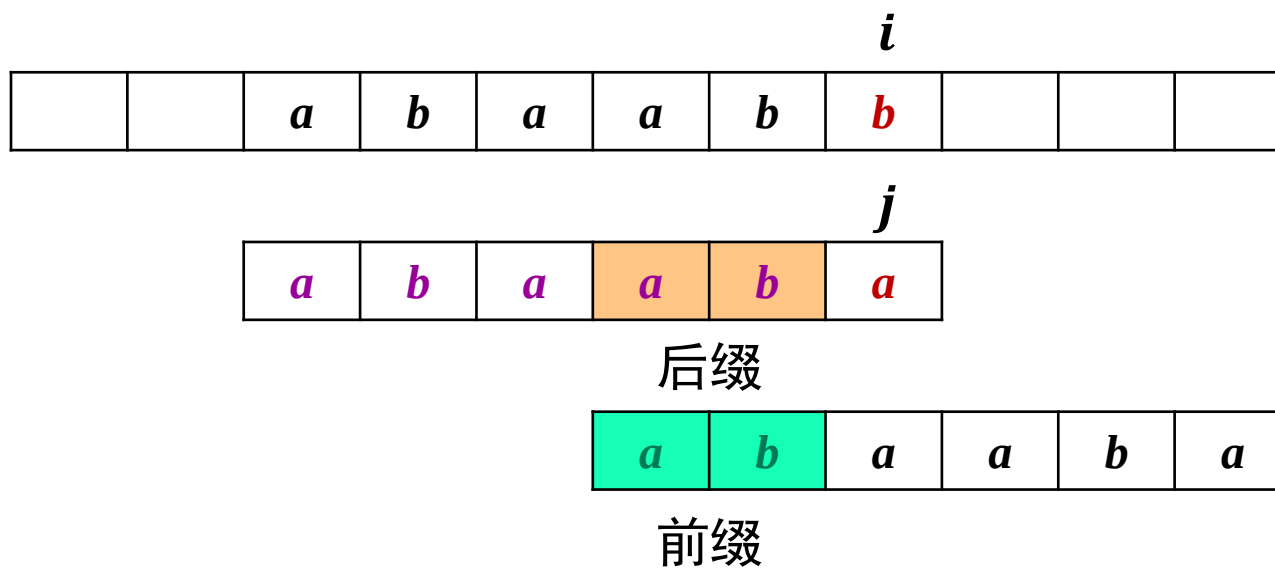
0		j			
a	b	a	a	b	a

- $P[0, \dots, j-2] = abaa$ ，其前缀为： a ， ab ， aba ， $abaa$
- $P[1, \dots, j-1] = baab$ ，其后缀为： $baab$ ， aab ， ab ， b
- 因此模式 P 在 $j-1$ 位置最长的匹配串为 ab ，称为部分匹配串，匹配串的长度为2



KMP算法

- 步骤二：每次发生失配时，根据最后一个匹配位置的部分匹配串长度向右移动模式串，使具有部分匹配长度的前缀对准相应的后缀所处的位置（利用模式串的信息以提高速度）
 - 每次失配发生时，将模式串向右移动尽可能远的距离，避免不必要的比较，提高匹配效率
 - 移动步数只与模式串 P 有关，而与目标串 T 无关





KMP算法

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

//模式串P的长度

```
    N[0] = 0;
```

//位置0处的部分匹配串长度为0

```
    int i = 1, j = 0;
```

//初始化比较位置

```
    while(i < m){
```

```
        if(P[i] == P[j]) {
```

//已经匹配了j+1个字符

```
            N[i] = j+1;
```

//部分匹配串长度加一

```
            i++; j++; }
```

//比较位置各进一

```
        else if( j > 0) j = N[j-1];
```

//移动：用部分匹配串对齐

```
        else { N[i++] = 0; }
```

//在串头时部分匹配串长度为0

```
    }
```

```
}
```



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
----------	----------	----------	----------	----------	----------	----------



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

m = 7

N[0] = 0

i = 0

<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
----------	----------	----------	----------	----------	----------	----------



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

i = 1, j = 0

i = 1

<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
----------	----------	----------	----------	----------	----------	----------

j = 0

<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
----------	----------	----------	----------	----------	----------	----------



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

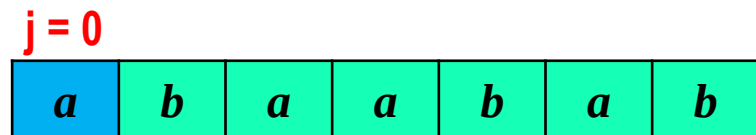
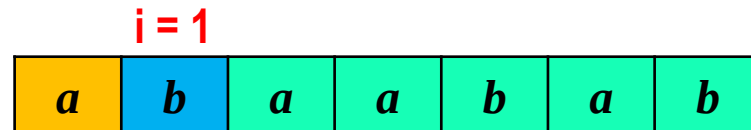
```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例



$N[1] = 0, i = 2$



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

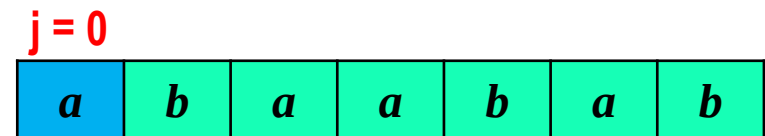
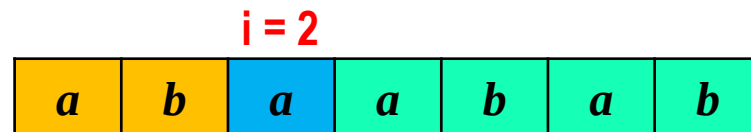
```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例



$N[1] = 0, i = 2$



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

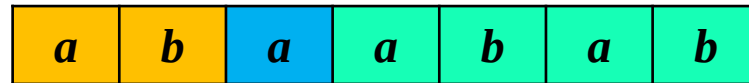
```
        else { N[i++] = 0; }
```

```
    }
```

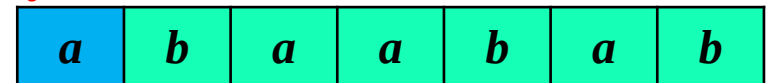
```
}
```

以模式串P = “*abaabab*” 为例

i = 2



j = 0



N[2] = 1

i = 3, j = 1



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

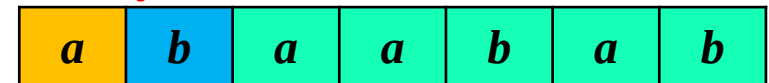
```
}
```

以模式串P = “*abaabab*” 为例

i = 3



j = 1



j = N[0] = 0



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

i = 3

a	b	a	a	b	a	b
---	---	---	---	---	---	---

j = 0

a	b	a	a	b	a	b
---	---	---	---	---	---	---

j = N[0] = 0



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){
```

满足

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

i = 3

a	b	a	a	b	a	b
---	---	---	---	---	---	---

j = 0

a	b	a	a	b	a	b
---	---	---	---	---	---	---

N[3] = 1

i = 4, j = 1



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){
```

满足

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

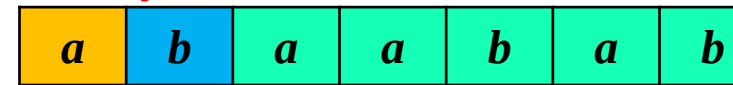
```
}
```

以模式串P = “*abaabab*” 为例

i = 4



j = 1



N[4] = 2

i = 5, j = 2



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){
```

满足

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

i = 5

a	b	a	a	b	a	b
---	---	---	---	---	---	---

j = 2

a	b	a	a	b	a	b
---	---	---	---	---	---	---

N[5] = 3

i = 6, j = 3



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

i = 6

a	b	a	a	b	a	b
---	---	---	---	---	---	---

j = 3

a	b	a	a	b	a	b
---	---	---	---	---	---	---

j = N[2] = 1



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

i = 6



j = 1



N[6] = 2

i = 7



KMP算法 - 过程分析

Next函数：计算模式串P每个位置部分匹配串的长度

```
void Next (char *P, int N[ ] ){
```

```
    int m = strlen(P);
```

```
    N[0] = 0;
```

```
    int i = 1, j = 0;
```

```
    while(i < m){ 不满足
```

```
        if(P[i] == P[j]) {
```

```
            N[i] = j+1;
```

```
            i++; j++; }
```

```
        else if( j > 0) j = N[j-1];
```

```
        else { N[i++] = 0; }
```

```
    }
```

```
}
```

以模式串P = “*abaabab*” 为例

N[0] = 0 N[2] = 1 N[4] = 2 N[6] = 2

a	b	a	a	b	a	b
---	---	---	---	---	---	---

N[1] = 0 N[3] = 1 N[5] = 3

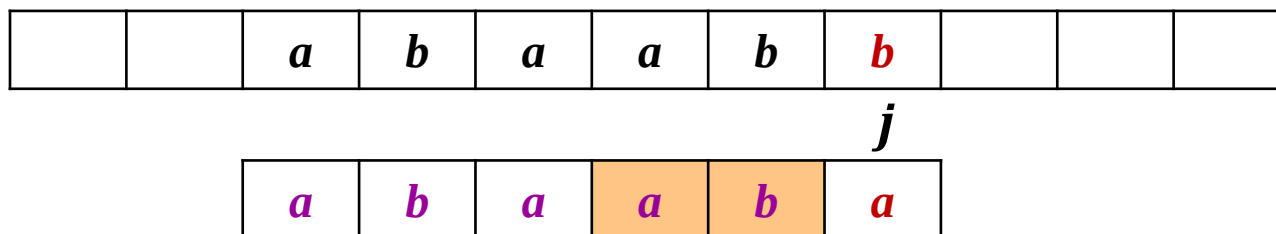
i = 7



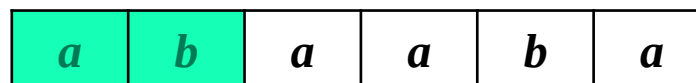
KMP算法

Next函数：计算模式串P每个位置部分匹配串的长度

- Next[j] 的值表示模式串P在j位置的部分匹配串长度
- 如果在模式串P的j位置出现失配，模式串向右移动；按照C语言中从0开始计数的习惯，模式串中Next[j - 1]位置就应该被移动到原来的j位置
- Next函数标志了失配后模式串下一个比较位置

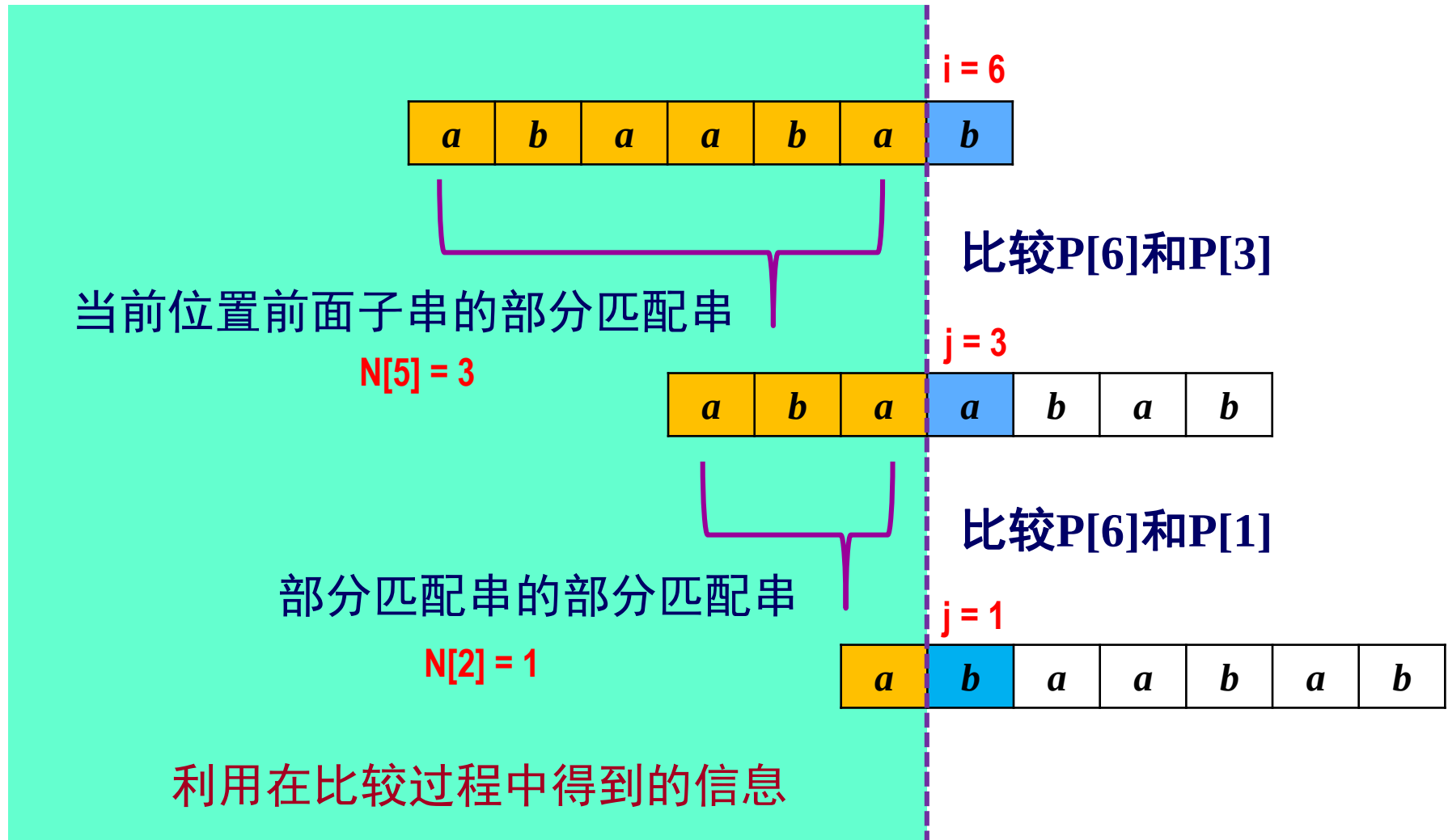


Next[j - 1] = **2**，下次比较P[**2**]





KMP算法 - 利用比较过程中得到的信息





KMP算法

```
int KMP_match(char *T, char *P){  
    int n = strlen(T), m = strlen(P);  
    int i=0, j=0, N[MaxSize];  
    next(P, N);  
    while (i<n) {  
        if (T[i]==P[j]) {  
            if (j == m-1) return i-j;  
            else { i++; j++; }  
        }  
        else {  
            if (j>0) j = N[j-1];  
            else i++;  
        }  
    }  
    return -1;  
}
```

//目标串和模式串长度

//预处理函数

//已经匹配 $j+1$ 个字符

//匹配成功，返回匹配位置

//比较下一个位置

//匹配不成功，移动到部分匹配串

//从后面的位置重新开始

//匹配失败



KMP算法 – 举例

<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>c</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>b</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>
----------	----------	----------	----------	----------	----------

<i>j</i>	0	1	2	3	4	5
$P[j]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>
$Next(j)$	0	0	1	0	1	2



KMP算法 – 举例

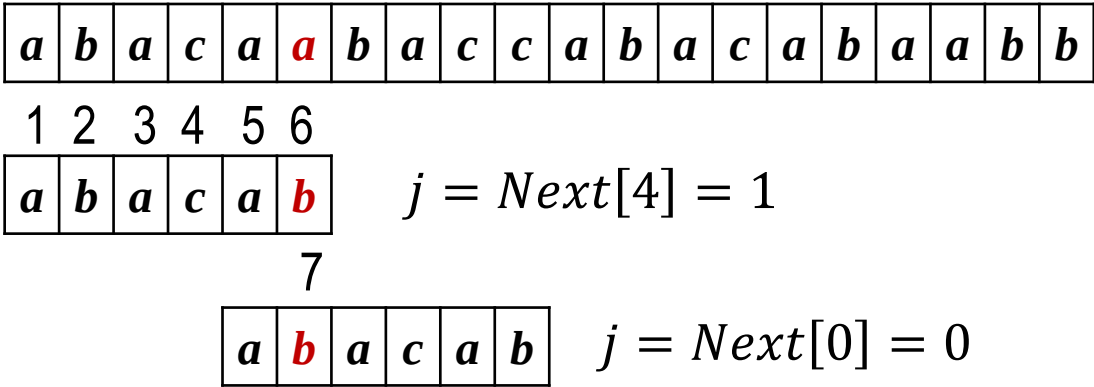
a	b	a	c	a	a	b	a	c	c	a	b	a	c	a	b	a	a	b	b
1	2	3	4	5	6														
a	b	a	c	a	a														

$j = \text{Next}[4] = 1$

j	0	1	2	3	4	5
$P[j]$	a	b	a	c	a	b
$\text{Next}(j)$	0	0	1	0	1	2



KMP算法 – 举例



<i>j</i>	0	1	2	3	4	5
<i>P</i> [<i>j</i>]	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>
Next(<i>j</i>)	0	0	1	0	1	2



KMP算法 – 举例

a	b	a	c	a	a	b	a	c	c	a	b	a	c	a	b	a	a	b	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1 2 3 4 5 6

a	b	a	c	a	b
---	---	---	---	---	----------

7

a	b	a	c	a	b
---	----------	---	---	---	---

8 9 10 11 12

a	b	a	c	a	b
---	---	---	---	----------	---

$j = \text{Next}[4] = 1$

$j = \text{Next}[0] = 0$

$j = \text{Next}[3] = 0$

j	0	1	2	3	4	5
$P[j]$	a	b	a	c	a	b
$\text{Next}(j)$	0	0	1	0	1	2



KMP算法 – 举例

a b a c a a b a c **c** a b a c a b a a b b

1 2 3 4 5 6

a b a c a **b**

$j = \text{Next}[4] = 1$

7

a **b** a c a b

$j = \text{Next}[0] = 0$

8 9 10 11 12

a b a c **a** b

$j = \text{Next}[3] = 0$

j	0	1	2	3	4	5
$P[j]$	a	b	a	c	a	b
$\text{Next}(j)$	0	0	1	0	1	2

a b a c a b

$j = 0; i++$



KMP算法 – 举例

a b a c a a b a c c a b a c a b a a b b

1 2 3 4 5 6

a b a c a b

$j = \text{Next}[4] = 1$

7

a b a c a b

$j = \text{Next}[0] = 0$

8 9 10 11 12

a b a c a b

$j = \text{Next}[3] = 0$

13

a b a c a b

$j = 0; i++$

14 15 16 17 18 19

a b a c a b

j	0	1	2	3	4	5
$P[j]$	a	b	a	c	a	b
$\text{Next}(j)$	0	0	1	0	1	2



KMP算法 – 时间复杂度分析

- KMP算法中while循环的每一步
 - 要么 i 加一 （目标串中的匹配点右移）
 - 要么移动量 $i - j$ 至少加一 （模式串右移）
- 因此KMP算法中while循环不会超过 $2n$ 次，计算复杂度是 $O(n)$
- **Next函数的建立方法与KMP算法一致**，Next函数中的while循环也不会超过 $2m$ 次
- 所以，KMP算法的时间复杂度是 $O(m + n)$
- 在最坏情况下，KMP算法的时间效率都要远好于蛮力算法



本讲小结

- 优先级队列
 - 一种特殊的线性结构
 - 根据是否维护优先级和存储结构，判断操作复杂度
- 串：概念与ADT、存储表示
 - 操作常以子串为单位进行
- 字符串的模式匹配
 - 蛮力法 vs. KMP算法（挖掘模式串信息，优化匹配过程）

加油同学们!

